Pre-launch security audit

broken-notes-app

Sample report. broken-notes-app is a small Next.js + Supabase + Stripe demo built with security holes on purpose (github.com/atokim4/supabase-scanner-bench). Every finding below is real scanner output on that code. It is not a client project.

7	3	1	1
CRITICAL	HIGH	MEDIUM	LOW



WHAT WAS CHECKED

- Supabase migrations: RLS, policies, views, SECURITY DEFINER functions, storage buckets.
- App code: server keys in browser code, public env prefixes, the Stripe webhook.
- Credentials in the files and in git history.
- Tools: vibe-preflight 1.3.0, secret-scan (built in) 10 patterns.

WHAT WAS NOT CHECKED

- The deployed site: no URL was given, so HTTPS, security headers, cookies and the built JavaScript bundles were not checked.
- Business logic and authorization inside your own API routes and server actions (beyond the Stripe webhook).
- The live database: policies or grants changed in the Supabase dashboard and not saved as migrations are invisible to a repo scan.
- Dependencies with known CVEs (run `npm audit` or Dependabot for that).
- Rate limiting, abuse and brute-force protection, email/auth settings in the Supabase dashboard.
- Manual penetration testing: every finding here comes from automated scanners and was not exploited against a running app.

Summary

The scan found **12 issues**: 7 critical, 3 high, 1 medium, 1 low. Fixing the 10 critical and high ones is about 7 to 22 hours of developer time, and 11 of the 12 come with a patch that applies to the code as it is.

THE THREE BIGGEST RISKS

1 **The master key to your database is sent to every visitor's browser.**

Critical · PA-01 · Server-only key used in browser code

2 **Anyone can fake a 'payment received' message and get paid features for free.**

Critical · PA-02 · Stripe webhook accepts unsigned requests

3 **A secret key (for example your Stripe key) is built into the public website code.**

Critical · PA-03 · Secret in a browser-exposed environment variable

ALL FINDINGS

ID	Severity	Issue	Where	Effort	Patch
PA-01	CRITICAL	Server-only key used in browser code	`app/admin/users-table.tsx:6`	1-4 hours	yes
PA-02	CRITICAL	Stripe webhook accepts unsigned requests	`app/api/stripe/webhook/route.ts:8`	under 1 hour	yes
PA-03	CRITICAL	Secret in a browser-exposed environment variable	`lib/stripe.ts:3`	under 1 hour	yes
PA-04	CRITICAL	Table open to the public API (Row Level Security off)	`supabase/migrations/20261101000700_audit_log.sql:2`	under 1 hour	yes
PA-05	CRITICAL	Privileged database function callable by anyone	`supabase/migrations/20261101001000_admin_set_plan.sql:13`	under 1 hour	yes
PA-06	CRITICAL	User accounts table exposed through a view	`supabase/migrations/20261101001100_user_directory.sql:2`	1-4 hours	yes
PA-07	CRITICAL	Public storage bucket	`supabase/migrations/20261101001200_invoices_bucket.sql:2`	1-4 hours	yes
PA-08	HIGH	Users can edit their own plan or role column	`supabase/migrations/20261101000100_profiles.sql:17`	1-4 hours	yes
PA-09	HIGH	View skips Row Level Security	`supabase/migrations/20261101000500_recent_notes.sql:2`	under 1 hour	yes
PA-10	HIGH	Policy lets everyone through (using true)	`supabase/migrations/20261101000600_messages.sql:15`	under 1 hour	yes
PA-11	MEDIUM	SECURITY DEFINER function without a fixed search_path	`supabase/migrations/20261101000900_message_stats.sql:2`	under 1 hour	yes
PA-12	LOW	Table has Row Level Security on and no policies	`supabase/migrations/20261101000800_feature_flags.sql:2`	under 1 hour	no

Findings

PA-01

CRITICAL

CWE
CWE-668

EFFORT
1-4 hours

SOURCE
vibe-preflight
server-secret-in-client

Server-only key used in browser code

app/admin/users-table.tsx:6

users-table.tsx runs in the browser and uses `SUPABASE_SERVICE_ROLE_KEY`.

```
   6 | const supabase = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.SUPABASE_SERVICE_ROLE_KEY!);
```

WHY IT MATTERS

Code in this file runs in the visitor's browser. A Supabase service role key or Stripe secret key there gives full control of the database or the Stripe account to anyone who opens developer tools. If it was ever deployed, treat the key as leaked.

FIX

Use the public anon key in the browser and keep the secret key in server code only. Rotate the key if this was ever deployed.

CWE-668: Exposure of Resource to Wrong Sphere

PATCH

The browser component now uses the public anon key (NEXT_PUBLIC_SUPABASE_ANON_KEY), so its queries run under Row Level Security as the signed-in user.

```
--- a/app/admin/users-table.tsx
+++ b/app/admin/users-table.tsx
@@ -3,7 +3,7 @@
 import { useEffect, useState } from "react";
 import { createClient } from "@supabase/supabase-js";

-const supabase = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.SUPABASE_SERVICE_ROLE_KEY!);
+const supabase = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!);

 type Profile = { id: string; display_name: string | null; plan: string };
```

Applies cleanly to the audited code (`git apply --check`).

- The component now sees only the rows its user may see. If it is an admin screen that needs every row, move the query to a route handler that checks the caller is an admin and uses the service role there.
- Rotate the service role key if this ever shipped.

PA-02

CRITICAL

CWE
CWE-345

EFFORT
under 1 hour

SOURCE
vibe-preflight
stripe-webhook-unverified

Stripe webhook accepts unsigned requests

app/api/stripe/webhook/route.ts:8

Handles Stripe events without `constructEvent`, so any POST is trusted.

```
   8 |   if (event.type === "checkout.session.completed") {
   9 |     const session = event.data.object;
  10 |     await supabaseAdmin
  11 |       .from("profiles")
```

WHY IT MATTERS

The webhook acts on whatever JSON it receives. Anyone who finds the URL can post a fake 'payment completed' event and get the paid plan for free.

FIX

Verify the `stripe-signature` header with `stripe.webhooks.constructEvent` on the raw request body before acting on the event.

CWE-345: Insufficient Verification of Data Authenticity

PATCH

Reads the raw body and checks the stripe-signature header with constructEventAsync before the handler touches the event. Unsigned or tampered requests get a 400.

```
--- a/app/api/stripe/webhook/route.ts
+++ b/app/api/stripe/webhook/route.ts
@@ -1,9 +1,24 @@
 import { NextResponse } from "next/server";
 import type Stripe from "stripe";
 import { supabaseAdmin } from "@/lib/supabase/admin";
+import { stripe } from "@/lib/stripe";

 export async function POST(request: Request) {
-  const event = (await request.json()) as Stripe.Event;
+  const signature = request.headers.get("stripe-signature");
+  if (!signature) {
+    return NextResponse.json({ error: "Missing stripe-signature header" },
{ status: 400 });
+  }
+
+  let event: Stripe.Event;
+  try {
+    event = await stripe.webhooks.constructEventAsync(
+      await request.text(),
+      signature,
+      process.env.STRIPE_WEBHOOK_SECRET!,
+    );
+  } catch {
+    return NextResponse.json({ error: "Invalid signature" }, { status: 400
});
+  }

   if (event.type === "checkout.session.completed") {
     const session = event.data.object;
```

Applies cleanly to the audited code (`git apply --check`).

- Set STRIPE_WEBHOOK_SECRET to the endpoint's signing secret (whsec_...) in every environment.
- Test with `stripe listen --forward-to localhost:3000/api/stripe/webhook` and `stripe trigger checkout.session.completed`.

PA-03

CRITICAL

CWE
CWE-668

EFFORT
under 1 hour

SOURCE
vibe-preflight
public-env-secret

Secret in a browser-exposed environment variable

`lib/stripe.ts:3`

`NEXT_PUBLIC_STRIPE_SECRET_KEY` looks like a secret and is exposed to the browser.

```
   3 | export const stripe = new
Stripe(process.env.NEXT_PUBLIC_STRIPE_SECRET_KEY!);
```

WHY IT MATTERS

Next.js copies every `NEXT_PUBLIC_` variable into the JavaScript it sends to visitors (Vite does the same with `VITE_`). A secret under that prefix is public as soon as the site is built.

FIX

Rename the variable without the public prefix, read it only in server code, and rotate the key if it was ever deployed.

CWE-668: Exposure of Resource to Wrong Sphere

PATCH

Renames NEXT_PUBLIC_STRIPE_SECRET_KEY to STRIPE_SECRET_KEY in .env.example, lib/stripe.ts, and marks lib/stripe.ts server-only so a browser import fails the build.

```
--- a/.env.example
+++ b/.env.example
@@ -1,6 +1,6 @@
 NEXT_PUBLIC_SUPABASE_URL=
 NEXT_PUBLIC_SUPABASE_ANON_KEY=
 SUPABASE_SERVICE_ROLE_KEY=
-NEXT_PUBLIC_STRIPE_SECRET_KEY=
+STRIPE_SECRET_KEY=
 STRIPE_WEBHOOK_SECRET=
 STRIPE_PRICE_ID=
--- a/lib/stripe.ts
+++ b/lib/stripe.ts
@@ -1,3 +1,4 @@
+import "server-only";
 import Stripe from "stripe";

-export const stripe = new
Stripe(process.env.NEXT_PUBLIC_STRIPE_SECRET_KEY!);
+export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);
```

Applies cleanly to the audited code (`git apply --check`).

- Rename the variable in your hosting dashboard too, and rotate the key if a build with NEXT_PUBLIC_ ever went live.

PA-04

CRITICAL

CWE
CWE-862

EFFORT
under 1 hour

SOURCE
vibe-preflight
rls-disabled

Table open to the public API (Row Level Security off)

`supabase/migrations/20261101000700_audit_log.sql:2`

`public.audit_log` is created without `enable row level security` in any migration.

```
   2 | create table public.audit_log (
   3 |   id bigint generated always as identity primary key,
   4 |   user_id uuid references auth.users (id) on delete set null,
   5 |   action text not null,
```

WHY IT MATTERS

Supabase serves every table in the public schema over its REST API, and the key it needs ships in your website. With Row Level Security off, anyone can read, change or delete every row of this table from a browser console.

FIX

Enable Row Level Security on the table. With no policies the API gets nothing, and your server code (service role) keeps working. Add policies only for what the browser really needs.

CWE-862: Missing Authorization

PATCH

Turns on Row Level Security for public.audit_log. With no policies the public API gets nothing; server code using the service role key keeps working.

```
--- /dev/null
+++ b/supabase/migrations/20261109090000_enable_rls_audit_log.sql
@@ -0,0 +1,3 @@
+-- PA-04: turn on Row Level Security for public.audit_log.
+-- Generated by prelaunch-audit. Review before applying.
+alter table public.audit_log enable row level security;
```

Applies cleanly to the audited code (`git apply --check`).

- If the browser needs to read or write this table, add policies for exactly those rows.

PA-05

CRITICAL

CWE
CWE-269

EFFORT
under 1 hour

SOURCE
vibe-preflight definer-function-anon

Privileged database function callable by anyone

`supabase/migrations/20261101001000_admin_set_plan.sql:13`

`public.admin_set_plan()` is SECURITY DEFINER and anon can call it at /rest/v1/rpc/admin_set_plan (granted explicitly).

```
  13 | grant execute on function public.admin_set_plan(uuid, text) to anon,
authenticated;
```

WHY IT MATTERS

The function runs with its owner's rights, so it skips Row Level Security. Supabase exposes it at /rest/v1/rpc/, and visitors who are not signed in may call it.

FIX

Revoke execute from `public`, `anon` and `authenticated`, grant it only to the roles that need it, and check the caller inside the function.

CWE-269: Improper Privilege Management

PATCH

Revokes execute on public.admin_set_plan() from public, anon and authenticated. The service role keeps its own grant, so server code can still call it.

```
--- /dev/null
+++ b/supabase/migrations/20261109090001_revoke_admin_set_plan.sql
@@ -0,0 +1,3 @@
+-- PA-05: stop the API roles from calling public.admin_set_plan().
+-- Generated by prelaunch-audit. Review before applying.
+revoke execute on function public.admin_set_plan(target_user uuid,
new_plan text) from public, anon, authenticated;
```

Applies cleanly to the audited code (`git apply --check`).

- Call it only from server code that has checked the user is allowed to (for an admin action: that they are an admin).

PA-06

CRITICAL

CWE
CWE-359

EFFORT
1-4 hours

SOURCE
vibe-preflight
auth-users-exposed

User accounts table exposed through a view

`supabase/migrations/20261101001100_user_directory.sql:2`

View `public.user_directory` selects from `auth.users` and the API serves it.

```
  2 | create view public.user_directory as
  3 |   select id, email, created_at
  4 |   from auth.users;
```

Also flagged by `view-bypasses-rls` on the same object.

WHY IT MATTERS

`auth.users` holds every account's email and sign-in data. A view over it in the public schema puts that list on the API, so anyone can download your user list.

FIX

Revoke API access to the view, and give users the lookup they need through a function that checks the caller and returns only safe columns.

CWE-359: Exposure of Private Personal Information to an Unauthorized Actor

PATCH

Revokes API access to public.user_directory, so the account list is no longer readable through /rest/v1/.

```
--- /dev/null
+++ b/supabase/migrations/20261109090002_revoke_user_directory.sql
@@ -0,0 +1,3 @@
+-- PA-06: take public.user_directory off the public API.
+-- Generated by prelaunch-audit. Review before applying.
+revoke all on public.user_directory from anon, authenticated;
```

Applies cleanly to the audited code (`git apply --check`).

- The feature that used this view stops working. Replace it with a function that checks the caller and returns only what it needs (for example the id for one exact email).

PA-07

CRITICAL

CWE
CWE-552

EFFORT
1-4 hours

SOURCE
vibe-preflight
storage-public-bucket

Public storage bucket

`supabase/migrations/20261101001200_invoices_bucket.sql:2`

Storage bucket `invoices` is public, and policy "Anyone can list invoices" lets anyone list every file in it.

```
  2 | insert into storage.buckets (id, name, public)
  3 | values ('invoices', 'invoices', true);
```

WHY IT MATTERS

Files in a public bucket open for anyone with the link. When a select policy on storage.objects also lets anyone list the bucket, the links are no secret either: anyone can download every file.

FIX

Make the bucket private, drop the open select policy, and hand out short-lived signed URLs from the server after checking who is asking.

CWE-552: Files or Directories Accessible to External Parties

PATCH

Makes the invoices bucket private and drops "Anyone can list invoices". Files stay where they are.

```
--- /dev/null
+++ b/supabase/migrations/20261109090003_private_bucket_invoices.sql
@@ -0,0 +1,4 @@
+-- PA-07: make the invoices bucket private.
+-- Generated by prelaunch-audit. Review before applying.
+update storage.buckets set public = false where id = 'invoices';
+drop policy if exists "Anyone can list invoices" on storage.objects;
```

Applies cleanly to the audited code (`git apply --check`).

- Links to these files stop working. Serve them with createSignedUrl() from server code after checking the file belongs to the user.

PA-08

HIGH

CWE
CWE-915

EFFORT
1-4 hours

SOURCE
vibe-preflight
rls-update-privileged-column

Users can edit their own plan or role column

`supabase/migrations/20261101000100_profiles.sql:17`

Policy "Users can update own profile" lets users update their row in `public.profiles`, including `plan` and `stripe_customer_id`. No column privilege or trigger stops them.

```
  17 | create policy "Users can update own profile"
  18 |   on public.profiles for update
  19 |   to authenticated
  20 |   using ((select auth.uid()) = id);
```

WHY IT MATTERS

The update policy lets a signed-in user change any column of their own row, including the one that says what they paid for. One request from the browser console turns a free account into a paid one.

FIX

Limit what users may update with column privileges (grant update only on the safe columns) and add `with check` to the policy. Change the plan only from the server, after Stripe confirms payment.

CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes

PATCH

Users can update only display_name; plan, stripe_customer_id can be changed only by the server. Adds a WITH CHECK to the policy.

```
--- /dev/null
+++ b/supabase/migrations/20261109090004_lock_columns_profiles.sql
@@ -0,0 +1,12 @@
+-- PA-08: users may only edit safe columns of public.profiles.
+-- Generated by prelaunch-audit. Review before applying.
+revoke update on public.profiles from anon, authenticated;
+grant update (display_name) on public.profiles to authenticated;
+
+drop policy "Users can update own profile" on public.profiles;
+
+create policy "Users can update own profile"
+  on public.profiles for update
+  to authenticated
+  using ((select auth.uid()) = id)
+  with check ((select auth.uid()) = id);
```

Applies cleanly to the audited code (`git apply --check`).

- If the app updates other columns from the browser, add them to the grant. Change paid columns only in server code (for example the Stripe webhook).

PA-09

HIGH

CWE
CWE-863

EFFORT
under 1 hour

SOURCE
vibe-preflight
view-bypasses-rls

View skips Row Level Security

`supabase/migrations/20261101000500_recent_notes.sql:2`

View `public.recent_notes` has no `security_invoker`, so it reads the tables below it without RLS.

```
  2 | create view public.recent_notes as
  3 |   select id, user_id, title, created_at
  4 |   from public.notes
  5 |   order by created_at desc
```

WHY IT MATTERS

Postgres views run with the rights of whoever created them, so they ignore the policies of the tables below. Supabase serves the view over the API, which gives every caller the rows your policies were meant to hide.

FIX

Set `security_invoker = true` on the view (Postgres 15+), so it runs with the caller's rights and the table policies apply.

CWE-863: Incorrect Authorization

PATCH

Sets security_invoker on public.recent_notes, so the policies of the tables below it apply to whoever queries it.

```
--- /dev/null
+++ b/supabase/migrations/20261109090005_security_invoker_recent_notes.sql
@@ -0,0 +1,3 @@
+-- PA-09: make public.recent_notes respect the caller's RLS.
+-- Generated by prelaunch-audit. Review before applying.
+alter view public.recent_notes set (security_invoker = true);
```

Applies cleanly to the audited code (`git apply --check`).

- Needs Postgres 15 or newer (every current Supabase project is).

PA-10

HIGH

CWE
CWE-863

EFFORT
under 1 hour

SOURCE
vibe-preflight
rls-permissive-policy

Policy lets everyone through (using true)

`supabase/migrations/20261101000600_messages.sql:15`

Policy "Enable read access for all users" on `public.messages` (select) lets anyone (signed in or not) read every row.

```
  15 | create policy "Enable read access for all users"
  16 |   on public.messages for select
  17 |   using (true);
```

WHY IT MATTERS

A policy with `using (true)` or `with check (true)` allows the action for every caller, including visitors who are not signed in. If the table holds anything private, it is public.

FIX

Replace the policy with one that compares `auth.uid()` to the owner column, and limit it to the `authenticated` role.

CWE-863: Incorrect Authorization

PATCH

Replaces "Enable read access for all users" with "Users can select rows they are in": signed-in users whose id is in sender_id or recipient_id.

```
--- /dev/null
+++ b/supabase/migrations/20261109090006_scope_policy_messages.sql
@@ -0,0 +1,8 @@
+-- PA-10: limit "Enable read access for all users" to the users in the
row.
+-- Generated by prelaunch-audit. Review before applying.
+drop policy "Enable read access for all users" on public.messages;
+
+create policy "Users can select rows they are in"
+  on public.messages for select
+  to authenticated
+  using ((select auth.uid()) = sender_id or (select auth.uid()) =
recipient_id);
```

Applies cleanly to the audited code (`git apply --check`).

- Check that sender_id / recipient_id really is who should have access. If the rows are meant to be public, keep the old policy and say so in a comment.

PA-11

MEDIUM

CWE
CWE-426

EFFORT
under 1 hour

SOURCE
vibe-preflight
definer-search-path

SECURITY DEFINER function without a fixed search_path

`supabase/migrations/20261101000900_message_stats.sql:2`

`public.get_message_stats()` is SECURITY DEFINER and doesn't set search_path.

```
   2 | create function public.get_message_stats()
   3 | returns table (sent bigint, received bigint)
   4 | language sql
   5 | stable
```

WHY IT MATTERS

The function runs with its owner's rights. Without a fixed `search_path`, a caller who can create objects in a schema on the path can make it run their code with those rights. Supabase's own linter flags this.

FIX

Add `set search_path = ''` to the function and schema-qualify every table it uses.

CWE-426: Untrusted Search Path

PATCH

Pins search_path to '' for public.get_message_stats(), so the function only sees the objects it names in full.

```
--- /dev/null
+++ b/supabase/migrations/20261109090007_search_path_get_message_stats.sql
@@ -0,0 +1,3 @@
+-- PA-11: pin the search_path of public.get_message_stats().
+-- Generated by prelaunch-audit. Review before applying.
+alter function public.get_message_stats() set search_path = '';
```

Applies cleanly to the audited code (`git apply --check`).

- Every table and function in the body must be schema-qualified (public.messages, auth.uid()).

PA-12

LOW

EFFORT
under 1 hour

SOURCE
vibe-preflight
rls-no-policy

Table has Row Level Security on and no policies

`supabase/migrations/20261101000800_feature_flags.sql:2`

`public.feature_flags` has RLS on and no policies: every API read and write is denied.

```
2 | create table public.feature_flags (
3 |   key text primary key,
4 |   enabled boolean not null default false
5 | );
```

WHY IT MATTERS

This is safe: the API returns nothing. But if the app reads this table from the browser, that read silently comes back empty, which looks like a bug at launch.

FIX

If the browser needs the table, add a read policy for the rows it should see. If only the server uses it, leave it and note that on purpose.

Appendix

HOW SEVERITY IS RATED

Each rule is placed by the first line that fits. Live-site findings keep the severity apify-launch-audit gives them.

CRITICAL	Anyone on the internet, without an account, can read private data, change data, take money-related actions or get a key that does those things.
HIGH	A signed-in user can reach other users' data or give themselves something they did not pay for, or private data is exposed but only under a condition the scanner cannot rule out.
MEDIUM	A missing safeguard that makes another bug easier to exploit or worse when it happens (headers, search_path, cookie flags), or a bug that breaks a feature in a way that fails closed.
LOW	Hygiene and launch polish: worth fixing, no direct path to an attack.

TOOLS

Tool	Version	What ran
vibe-preflight	1.3.0	`node vendor/vibe-preflight/preflight.mjs broken-notes-app --format json --fail-on never`
secret-scan (built in)	10 patterns	`working tree (git ls-files) + git log -p --all`

REPRODUCE

```
prelaunch-audit broken-notes-app
```

APPLY THE PATCHES

Each patch is in `patches/<id>.diff`. From the root of the repository: `git apply --check patches/PA-01.diff`, then `git apply patches/PA-01.diff`. New migrations go through `supabase db push` as usual. Read every patch first: they are written from the code as it was scanned and cannot know your intent.

REPO RULES AND THEIR CWE

Rule	Issue	CWE
`rls-disabled`	Table open to the public API (Row Level Security off)	CWE-862
`rls-permissive-policy`	Policy lets everyone through (using true)	CWE-863
`rls-no-policy`	Table has Row Level Security on and no policies	none
`rls-update-privileged-column`	Users can edit their own plan or role column	CWE-915
`view-bypasses-rls`	View skips Row Level Security	CWE-863
`auth-users-exposed`	User accounts table exposed through a view	CWE-359
`definer-search-path`	SECURITY DEFINER function without a fixed search_path	CWE-426
`definer-function-anon`	Privileged database function callable by anyone	CWE-269
`storage-public-bucket`	Public storage bucket	CWE-552

Rule	Issue	CWE
`server-secret-in-client`	Server-only key used in browser code	CWE-668
`public-env-secret`	Secret in a browser-exposed environment variable	CWE-668
`stripe-webhook-unverified`	Stripe webhook accepts unsigned requests	CWE-345
`stripe-webhook-parsed-body`	Stripe signature checked against a parsed body	CWE-345
`env-file-committed`	Environment file with values in git	CWE-312
`secret-in-tree`	Credential in the repository	CWE-798
`secret-in-history`	Credential left in git history	CWE-798